

Logical Agents:

On Knowledge Representation Power:

→ In AI: the approach of reasoning with knowledge is embodied in Knowledge-based agents.

* Previous types of agents: allowed to move using domain specific functions, whereas now: they can reason.

→ We used to list everything (because it's the only choice for the agent).

* CSP → some parts of the agent work in a domain-independent way → allowing for more efficiency.

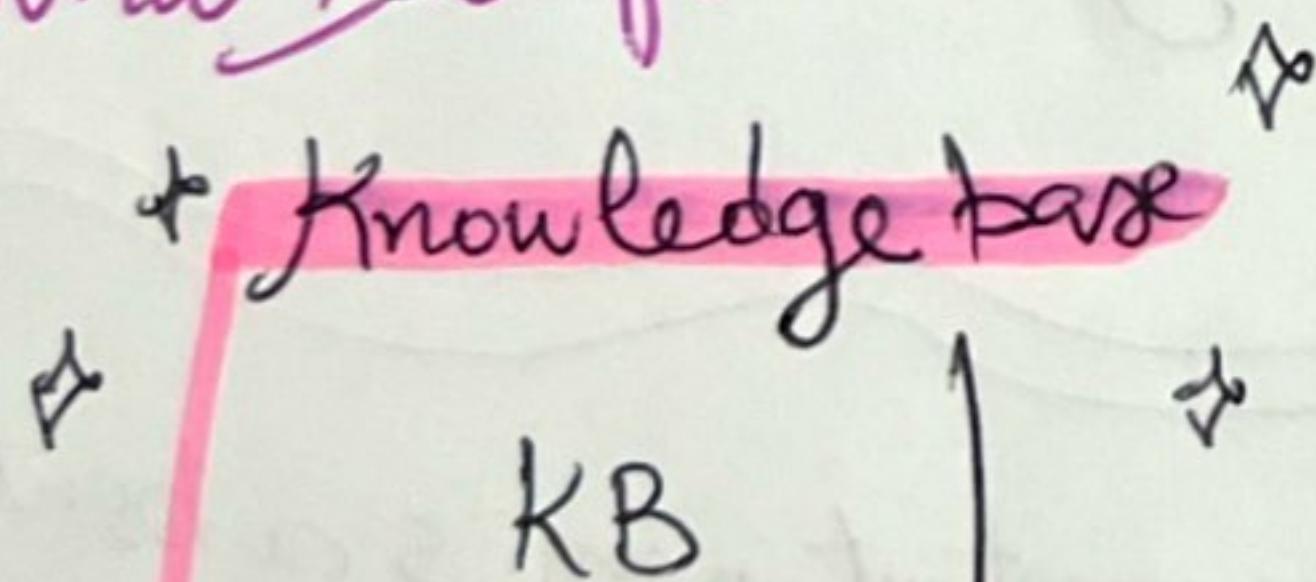
→ Logic will be used to support Knowledge based agents

* The combination of knowledge allowing for reasoning (inference)

Knowledge based agents: Can:

- ① Accept new tasks in the form of explicit goals.
- ② Achieve competence quickly by being told or learning new knowledge about the environment.
- ③ Adapt to changes by updating the relevant knowledge.

Central Component:



Functions:

TELL

way to add new sentences to the KB

ASK

way to query the KB.

they both involve deriving new sentences from old ones.

Axiom:

is a sentence taken as Given, without being derived from other sentences.



Details of representation language are hidden in the 3 functions: and details of inference are hidden in TELL ASK

function KB-AGENT(percept) returns an action

persistent: KB, a knowledge base
t counter for tie.

TELL(KB, MAKE-PERCEPT-SENTENCE(percept, t));

action = ASK(KB, MAKE-ACTION-QUERY(t))

TELL(KB, MAKE-ACTION-SENTENCE(action, t))

asks what action it should perform.

(after reasoning about current state...)

Tells the KB which action was chosen and the agent executes it

t = t + 1

return action

* Declarative Vs procedural *

is stating the knowledge

At Knowledge Level: the agent will add more and more until it knows how to operate in its environment.

> Declarative Approach.

(we don't care that much about implementation level).

> Procedural Approach: encodes desired behaviours directly as Program Code

Wumpus World

• Illustrates some important points about intelligence.

• The wumpus can be shot by an agent, but has only one arrow.

• Describing it as PEAS:

■ Performance measure: some rules for each action.
+1000 for clinching.
-10 using the arrow...

■ Environment: 4x4 Grid of rooms.
Description of the world

Actuators: Agent can move: Forward, Turn Left by 90° , Turn Right by 90° .

- Agent dies if it enters a square with a pit / or liveswamp.
- Each action what will do...

Sensors: It has 5 sensors: each of them give one information.

• Either perceive: Stench / Breeze / Glitter / Bump / Scream.
and this will be given to the agent program in the form of a list of 5 symbols.

[None, None, None, None, None].
Stench Breeze Glitter Bump Scream.

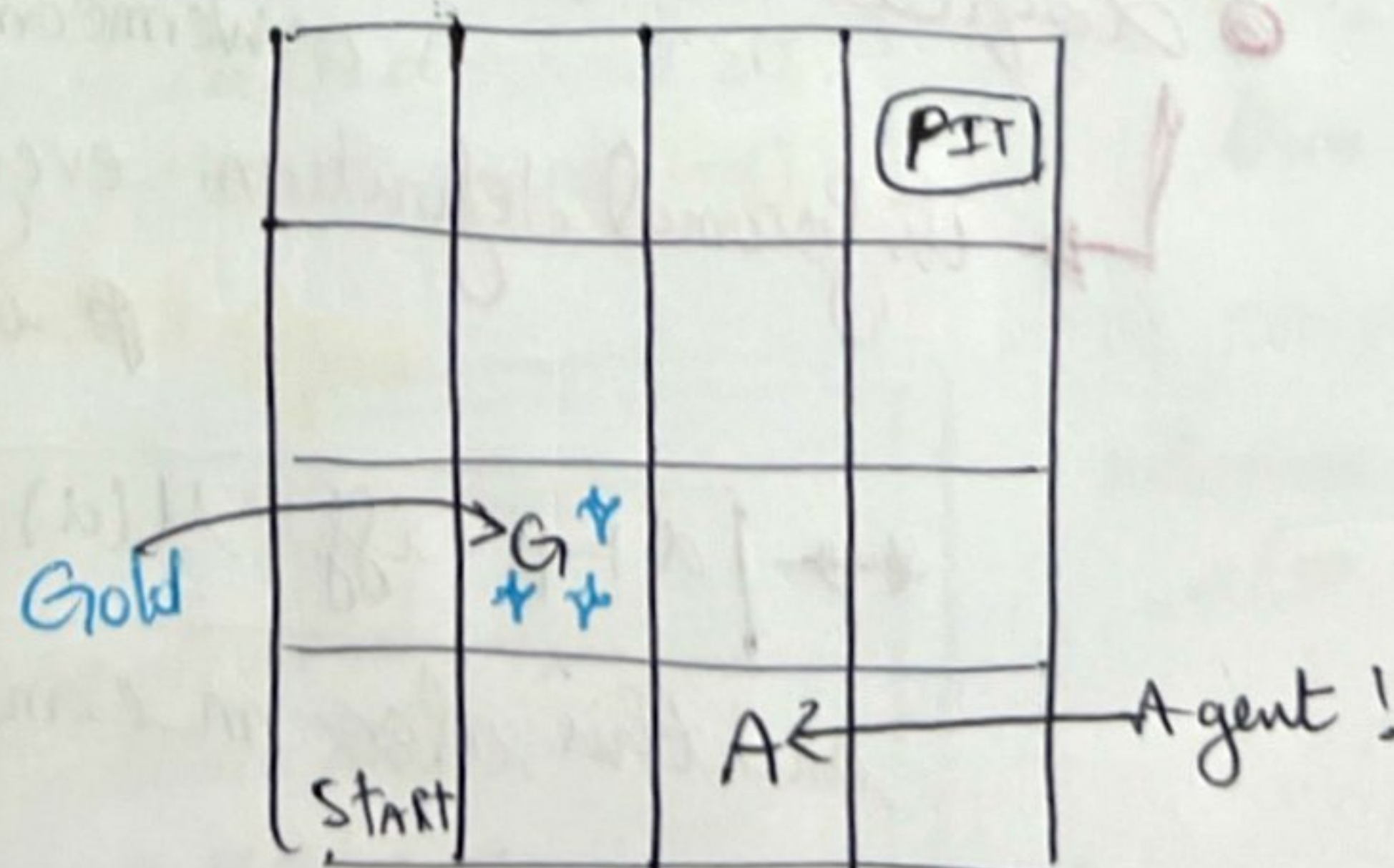
Reasoning: for the Wumpus World.

Main challenge: its initial ignorance of the configuration of the environment.
↳ We need logical reasoning to overcome it!

The agent's initial KB: contains the rules of the environment

• Depending on the percept it will know.

• Informal knowledge representation language is required and used for writing down symbols in a grid.



+++ Logic:

- Sentences in the KB are expressed according to Syntax of the representation language.
(it will specify all sentences that are well formed)

↓
Defines Semantics: meaning of sentences.

↳ they define the truth of each sentence with respect to each possible world.

+Eg: the semantics for arithmetic specifies that:

" $x+y=4$ " is true in a world where $x=2, y=2$
but false where $(1,1)$.

Remark:

→ In Standard Logics: every sentence must be true or false!! ^{either}

Model is used to descr in place of "possible world"

• For example: " $x+y=4$ " possible models are all possible assignments of real numbers to the variables x and y .

• each assignment fixes the truth of any sentence of arithmetic whose variables are x and y .

* Set of models of α is: $M(\alpha)$

[• If sentence α is true in model m .
then we say: m satisfies α : m is a model of α

• Logical entailment: " $\alpha \models \beta$ " states " α entails β "

or we mean: β logically follows from α

↳ its formal definition: every model in which α is True,
→ β is also true

$$\alpha \models \beta \text{ iff } M(\alpha) \subseteq M(\beta)$$

and this also means that: α is a stronger assertion than β and it rules out more possible worlds.

eg: $(x=0) \models (xy=0)$

* The KB is false in models that contradict the agent's knowledge.
(We can find a finite set of models (eg 3) in which KB is true).

◦ We have in our KB that $[1,1]$ does not contain breeze
So we can't have a pit in $[1,2]$.

We can say then:

α_1 : "there is no pit in $[1,2]$ "

$\boxed{KB \models \alpha_1}$ it means: every model in which
KB is true, α_1 is also true.
by inspection.

α_2 : "No pit in $[2,2]$ "

$\alpha_2 = \text{false}$ $\rightarrow$ in some models of KB, α_2 is false.
hence $KB \not\models \alpha_2 \rightarrow$ We cannot conclude that.

LOGICAL ENTAILMENT $\rightarrow$ used to carry out logical inference. and Inference:

• Model checking: enumerating all possible models to check that α is true in all models in which KB is true:

$$M(KB) \subseteq M(\alpha)$$

• Inference algorithm: i can derive α from KB: " $KB \vdash_i \alpha$ "
" α is derived from KB by i "

if this algo derived only
entailed sentences: then
it is called sound or
(truth-preserving)

i is sound: if whenever:
 $KB \vdash_i \alpha$, it is also
true that $KB \models \alpha$.

i is complete: if
whenever $KB \models \alpha$, it is
also true that
 $KB \vdash_i \alpha$.

Remark: if KB is true
in the real world, then
any sentence α derived
from KB by a sound
inference procedure
is also true in the
real world

5

Propositional logic Syntax: allowable sentences in any language.

- **Atomic sentence:** single proposition symbol, can be T/F they start with **uppercase letters**.
- **Complex sentences:** constructed from **simpler sentences** using **logical connectives** and **parenthesis**.

BNF of propositional logic

"Backus-Naur-form"

Sentence $\rightarrow$ Atomic | complex

Atomic $\rightarrow$ True | False | P | q

Complex $\rightarrow$ (sentence) | [sentence]

----- page 27

• and logical operators have precedence from:

$\neg \wedge \vee \Rightarrow \leftrightarrow$
Highest priority

there are 5 connectives:

• $\neg$ (not): $\neg W_{1,3}$ is negation of $W_{1,3}$

• **literal:** atomic sentence (+ literal)
 or **negated** atomic sentence (- literal)

• **and** " $\wedge$ ": conjunction of conjuncts

• **or** " $\vee$ ": disjunction of disjuncts

• $\leftrightarrow$ (iff): a sentence connected by **biconditional symbol**

• $\rightarrow$ (implies): implication or conditional, its **premise or antecedent** is "A" and its **conclusion or consequent** is "B"
 $A \rightarrow B$

Propositional logic Semantics:

- they define the rules for determining the truth of a sentence with respect to a particular **model**
- Model fixes the **truth value** (T/F) of every proposition symbol
- Use "a truth table": Semantics for prop. logic: how to compute the truth value of any sentence given a model (**done recursively**)



A simple knowledge base:

Example in the Wumpus World:

Representation of its KB:

① Symbols for each location:

$P_{x,y}$: if there is a pit in (x,y) . / $W_{x,y}$ / $B_{x,y}$ / $S_{x,y}$.

← wumpus
← breeze
← stench

② Sentences: Each sentence R_i is labelled for referencing purposes.

eg: There is no pit in $[1,1]$: $R_1: \neg P_{1,1}$...

③ Decide whether: $KB \models \alpha$? (for some α)

* First inference Algorithm: model checking approach:

(Construct the truth table)

- ① • enumerate the models
- ② • Check that α is true in every model which KB is true

(direct implementation of entailment definition)

• We have the truth-table enumeration algorithm for deciding propositional entailment. Slide 34

Properties of TT-entails algorithm.

• it is: • Sound: because it implements directly the def of entailment.

• Complete: it works for any KB and α and always terminates (there are only finitely many models to examine and enumerates them).

* Complexity of TT-entails:

• If KB and α contain n symbols in all, then there are 2^n models $\rightarrow$ time complexity is: $O(2^n)$

• Space Complexity: $O(n)$ enumeration is depth first

standard logical equivalences (in slide 36)

* Propositional theorem proving:

- How entailment can be done by theorem proving: applying rules of inference directly to sentences in KB to construct a proof of a given sentence without consulting models.
- More efficient when there is a large number of models and proof is short.

- Use Logical Equivalence: 2 sentences α and β are logically equivalent ($\alpha \equiv \beta$) : if they are true in the same set of models.
 $\iff$ if they entail each other:

$$\alpha \equiv \beta \iff (\alpha \models \beta) \wedge (\beta \models \alpha)$$

* Validity and Satisfiability:

- Sentence is valid: true in all models, "tautology" and validity is connected to inference via the deduction theorem $KB \vdash \alpha$ iff $(KB \Rightarrow \alpha)$ is valid.

- Satisfiable: True for some model:

- Unsatisfiable: True for no Model

can be checked by enumerating the possible models until one is found!

Problem of determining Satisfiability of sentences known as the SAT problem

- And many CS problems are SAT. (like in CSP)

- Some remarks: α is valid iff $\neg \alpha$ is unsatisfiable

Results: α is satisfiable iff $\neg \alpha$ is not valid

- $KB \models \alpha$ iff $(KB \wedge \neg \alpha)$ is unsatisfiable

$$\neg (KB \models \alpha)$$

Proving α from KB by checking the unsatisfiability of $(KB \wedge \neg \alpha)$ = Proof by contradiction / refutation.

Inference and Proofs:

(1) Model checking
as we saw previously.

(2) Application of Inference rules

• **Inference rule:** rule that can be applied as 1 step in a proof i.e. in a chain of conclusions that leads to the desired goal.

Example: Modus Ponens

$$\frac{\alpha \rightarrow \beta \quad \alpha}{\beta}$$

• Infer shoot from
" (WAhead & WAlive)
→ shoot"

And elimination:

$$\frac{\alpha \wedge \beta}{\beta}$$

• Consider truth values of α and β → to show Modus Ponens and And-elimination are **Sound**

• All logical equivalences can be used as **inference rules!**

they are equivalent to operators in standard search algorithm.

→ and it typically requires them to be translated into Normal form.

"Examples slide 42"

Proof as Search:

it can be defined:

- **INITIAL state:** the initial KB
- **ACTIONS:** set of actions, consists of all the **inference rules** applied to all the sentences that match the top half of inference rule.
- **RESULT:** add the sentence in the bottom half of the inference rule to the KB
- **Goal:** State that contains the sentence we are trying to prove.

(More efficient because - proof can ignore irrelevant propositions)

Alternative to enumerating models

19

♦ Monotonicity: Final property of logical systems:
 $\hookrightarrow$ "set of entailed sentences can only increase as information is added to the KB"

♦ We have: for any sentence: a and b if $KB \models a$,
 then $KB \wedge b \models a$

♦ an additional assertion b can never invalidate any conclusion a already inferred, the agent might be helped to draw additional conclusions

In short: this means that inference rules can be applied whenever suitable premises are found in the KB

(once you have the condition)
 $\rightarrow$ draw your conclusion

$\rightarrow$ The conclusion of the rule must follow, regardless of what else is in the KB!

PROOF by Resolution

♦ Resolution: A single inference ~~algorithm~~ rule that yields a complete inference algorithm when coupled with any search algorithm.

$\rightarrow$ Its application: when we have: $\neg P_{2,2}$ in R_3 that

Example of the unit resolution

resolves with the literal $P_{2,2}$ to

give $P_{1,1} \vee P_{3,1}$

$\boxed{\neg P_{2,2} \vee P_{2,2}}$ will go.

m and l_i are complementary to each other.
 (they negate each other)

$$\frac{l_1 \vee l_2 \vee \dots \vee l_m}{l_1 \vee \dots \vee l_{i-1} \vee l_{i+1} \vee \dots \vee l_m}$$

$\hookrightarrow$ Generalisation to: the full resolution rule:

• Clause: is a disjunction of literals

• Unit resolution rule: takes a clause and a literal and produces a new clause

$$\frac{l_1 \vee \dots \vee l_m \quad m_1 \vee \dots \vee m_n}{l_1 \vee \dots \vee l_{i-1} \vee l_{i+1} \vee \dots \vee l_m \vee m_1 \vee \dots \vee m_{j-1} \vee m_{j+1} \vee \dots \vee m_n}$$

* The resolution inference rule: is "Sound", and "Complete" because of the completeness of resolution and for any $KB \models \alpha$, decides if $KB \models \alpha$

CNF: Conjunctive Normal Form:

- Resolution is only on disjunctions!
- Completeness is ensured, because every sentence of propositional logic is logically equivalent to a conjunction of clauses!

* Converting to CNF: | ① Eliminate $\leftrightarrow$ by $(A \leftrightarrow B) \text{ by } (A \rightarrow B) \wedge (B \rightarrow A)$
| ② Replace $A \rightarrow B$ with $\neg A \vee B$

Example: $B_{1,1} \leftrightarrow (P_{1,2} \vee P_{2,1})$

$$(B_{1,1} \rightarrow (P_{1,2} \vee P_{2,1})) \wedge B_{1,1} ((P_{1,2} \vee P_{2,1}) \rightarrow B_{1,1})$$

$$((\neg B_{1,1}) \vee (P_{1,2} \vee P_{2,1})) \wedge ((\neg(P_{1,2} \vee P_{2,1}) \vee B_{1,1}))$$

$$\leftrightarrow (\neg B_{1,1} \vee P_{1,2} \vee P_{2,1})$$

$$\wedge ((\neg P_{1,2} \wedge \neg P_{2,1}) \vee B_{1,1})$$

$$\leftrightarrow (B_{1,1} \vee (\neg P_{1,2})) \wedge (B_{1,1} \vee \neg P_{2,1})$$

$$\wedge (\neg B_{1,1} \vee P_{1,2} \vee P_{2,1})$$

| ③ use de Morgan Laws + equivalences (move $\neg$ inwards).

| ④ Distribute $\vee$ over $\wedge$:

a Resolution Algorithm

• Inference procedures based on resolution work by using Proof by contradiction.

• To show $KB \models \alpha$ is true, show that $(KB \wedge \neg \alpha)$ is unsatisfiable! $\rightarrow$ proof by contradiction.

* La procedure:

Resolution
Algo
slide 5 & 2

- ① Convert to CNF
- ② Resolution is applied to resulting clauses, and each pair that contains complementary literals is **resolved** → produce new clause
- ③ Add new clause to KB, if not there.
- ④ Continue Process until one of 2 things happen.
 - ↙ No new clauses that can be added (case KB $\neq \emptyset$)
 - ↘ 2 clauses resolve to yield empty clause in which KB entails $\perp$

* **An empty clause:** Disjunction of no disjuncts is equivalent to **False** because: a disjunction is true only if one of its disjuncts is True.

it represents a contradiction

Since it arises only from resolving 2 complementary unit clauses such as P and $\neg P$.

Example of a proof slide 53
we reach a contradiction
□ so TP is True.

* Horn Clauses and definite clauses:

Resolution is **complete** → important inference method

(there are more efficient)

In many practical situations, its power is not needed.

Definite clause: Disjunction of literals of which **exactly one** is positive!

$\neg B_{1,2} \vee P_{1,2} \vee \neg P_{2,2}$ is not

Horn Clause: at most one is positive (slightly more general than definite)

Closed under resolution

- All definite clauses $\rightarrow$ Horn Clauses.
- Clauses with no $\oplus$ literals $\Rightarrow$ Goal clauses.

KBs of Horn Clauses have advantages:
(body $\Rightarrow$ head)

examples $\leftarrow$

S1. $\neg L_{1,1} \vee \neg B \vee B_{1,1}$

S2. $\neg (L_{1,1} \wedge B) \vee B_{1,1}$

S3. $(L_{1,1} \wedge B) \Rightarrow B_{1,1}$
Body Head

(+) Every Definite Clause can be written as:

$(P_1 \wedge P_2 \wedge \dots \wedge P_k) \rightarrow P_{\text{positive}}$
Conjunction of positive literals

• Sentence with a single positive literal is called a **Fact**
True $\Rightarrow L_{1,1}$

(+) Inference with **Horn clauses**: can be done with forward chaining and backward chaining algorithms which are: the basis of **logic programming**

(+) Deciding **Entailment** with Horn clauses can be done in time "linear" in the size of the knowledge base

Forward Chaining Algorithm.

PL-FC ENTAILS? (KB, q)
is query "q" entailed by KB definite clauses??

13

* Begins from: Known facts: $\oplus$ literals in the "KB"

* If the premises of an implication are known $\rightarrow$ added to the set of Known facts.

* Process continues until q is added or no further inferences can be made.

⚠ * Runs in line or time!

* It is Sound and Complete

every inference is essentially an application of Modus Ponens

every entailed atomic sentence will be derived.

Complete algo in slide 60

* FC being is an example of the general concept: data driven reasoning.

* It can be used without a specific query! in mind
 $\hookrightarrow$ agent will derive conclusions from incoming percepts.

focus of attention is our known data.

data driven - Reasoning

with humans: once new info arrives:

like when you hear rain

$\rightarrow$ cancel picnic.



BACKWARD CHAINING:

- Works backward from the query.
→ if query q is known to be true, then **Now work needed**
- Otherwise, the Algorithm finds those implications in the KB whose conclusion is q .
- All premises of one of those implications can be proved to be true $\Rightarrow$ then q is true.
(by backward)

• Example in Slide 64

Moves backward until we reach known facts A and B.

$P \Rightarrow Q \leftarrow \checkmark$
 $L \wedge M \Rightarrow P$
 $B \wedge L \Rightarrow M$
 $A \wedge P \Rightarrow L$
 $A \wedge B \Rightarrow L$
 $A \checkmark$
 $B \checkmark$



has a correspond
AND-OR GRAPH.

- Backward chaining: **Identical to AND-OR Search** Graph.
- An efficient implementation of BC runs in linear time.

• It is a form of **Goal-directed Reasoning** useful for answering questions like:

"Is the battery faulty?"

"Can I cook this dish?"

- It touches only relevant Facts
→ so it is **much less than linear** in size of the KB.

AGENTS BASED ON PL:

Example
on
construction
the Agent
in
the
Wumpus
World!
Slide 67.

Step 01: Given its percept history → Agent can deduce all it can about the state of the world.

Step 02: Write down a complete logical model of the effects of actions.

Step 03: How it will be deducing its current state of the wumpus world.

• depending on assumptions → we eliminate

Step 04: More specification:

MAKE-REPRESENTATION [associate the steps to percepts.
to avoid contradictions.]

Remark: • No need for timestamps: "Atemporal Variables"

✦ Fluent: aspect of the world that changes.

↳ • Connect properties ...

✦ Need for AXIOMS that allow the agent to keep track of fluents

Step 05: Write down Transition Model as a set of logical sentences that capture fluents changes as the result of the agent's actions.

Step 06: Effect AXIOMS: outcome at the next
the step!
• Need to do it for each possible time step,
for each 16 squares, each of the four
orientations.



represented
frame problem

↓
real world has many fluents!

• We get the frame problem: when we don't know what remains unchanged
Possible solution: add frame axioms asserting all the propositions that remain the same.

Drawback:

- Inefficient.
- size would be $O(mn)$ = Set of frame axioms.
↑
actions

• Each action changes k of total fluents → the world exhibits: called locality.

→ need to define a transition model with a size $O(mk)$ rather than $O(mn)$ as set of axioms.

Problem 2: Inferential frame problem: projecting the result forward.

↓
Solution: change one's focus to write axioms about fluents.

Successor-state Axiom:

of the form $F^{t+1} \leftrightarrow \text{Action Cause } F^t \vee (F^t \wedge \neg \text{Action})$.
the schema of this axiom.

Given all: complete

- set of successor state axioms
- the other axioms listed

→ Agent will be able to ask, and

ANSWER about current state.

↗ Last serious problem:

→ We need to confirm all necessary preconditions of an action hold ~~to~~ to have its effect.

↳ Many unusual exceptions → Action will fail.

Specify all these exceptions. qualification problem

